

Paradigmi di programmazione

Contents

1	Introduzione	3
2	I primi paradigmi	4
3	Descrizione dei paradigmi	5
3.1	Paradigma imperativo	5
3.2	Paradigma object-oriented	5
3.3	Paradigma funzionale	5
3.4	Programmazione concorrente	5
3.5	Programmazione parallela	5
4	Il ruolo chiave dei tipi	6
5	Linguaggi	7
5.1	Fortran	7
5.2	ALGOL 60	7
5.3	LISP	7
5.4	SIMULA 67	7
5.5	C	8
5.6	Pascal	8
5.7	Smalltalk	8
5.8	PROLOG	8
5.9	C++	8
5.10	Java	8
5.11	Altri linguaggi popolari	9
5.12	Come crescono i linguaggi?	9
6	Approccio scientifico	10
7	Lambda Calculus	11
7.1	Storia della calcolabilità	11
7.2	Macchina di Turing	11
7.3	Architettura di Von Neumann	11

1 Introduzione

Email Docente: chiara.bodei@unipi.it

E-Learning: <https://elearning.di.unipi.it/course/view.php?id=1167> Il Teams è privato, non si può entrare. (Future notice)

Lo scopo del corso è di facilitare l'apprendimento di nuovi linguaggi di programmazione attraverso lo studio dei paradigmi.

2 I primi paradigmi

Una volta i linguaggi di programmazione si programmavano in modo molto diverso dall'altro, ma oggi seguono appunto delle regole e design pattern comuni.

Per esempio:

- mancano altri
- **Fortran** utilizzava un paradigma **imperativo** (verb-oriented): il programma è una successione di comandi di assegnamento, si basa su uno **stato** rappresentato da un insieme di variabili che si aggiornano fino ad uno stato finale in cui viene generato il risultato.

3 Descrizione dei paradigmi

Non esiste un paradigma migliore degli altri: la scelta dipende dalla situazione e dal problema da risolvere.

Molti programmi possono utilizzare, per sottoproblemi diversi, approcci differenti (**multiparadigma**).

3.1 Paradigma imperativo

Il **paradigma imperativo** descrive il programma come una sequenza di istruzioni che modificano lo stato del programma.

Il programmatore specifica **come** ottenere il risultato, indicando esplicitamente le operazioni da eseguire.

Esempi di costrutti tipici sono assegnamenti, cicli e istruzioni condizionali.

3.2 Paradigma object-oriented

Il **paradigma object-oriented** (o **a oggetti**) organizza il programma attorno a oggetti che combinano **dati** e **comportamenti**.

Gli oggetti appartengono a **classi**, che ne definiscono caratteristiche e operazioni.

I principali concetti sono **incapsulamento**, **ereditarietà**, **polimorfismo** e **astrazione**.

3.3 Paradigma funzionale

Il **paradigma funzionale** considera il programma come una composizione di **funzioni**, cercando di evitare modifiche dello stato e effetti collaterali.

Le funzioni possono essere trattate come valori, passate come argomenti e restituite come risultati.

Sono importanti concetti come **immutabilità**, **funzioni pure**, **ricorsione** e **funzioni di ordine superiore**.

3.4 Programmazione concorrente

La **programmazione concorrente** permette a più attività di essere **in esecuzione nello stesso intervallo di tempo**, anche se non vengono necessariamente eseguite simultaneamente.

Le attività possono alternarsi e comunicare o sincronizzarsi tra loro.

È utile, ad esempio, per gestire contemporaneamente richieste di utenti, operazioni di I/O o attività indipendenti.

3.5 Programmazione parallela

La **programmazione parallela** consiste nell'eseguire più operazioni **simultaneamente**, generalmente sfruttando più processori o core.

L'obiettivo principale è ridurre il tempo necessario per completare un'elaborazione, dividendo il problema in parti che possono essere eseguite in parallelo.

La programmazione parallela è quindi particolarmente utile per problemi scomponibili in attività indipendenti o parzialmente indipendenti.

4 Il ruolo chiave dei tipi

I **tipi** nella programmazione sono molto importanti, in quanto dichiarano gli intenti di dati e funzioni.

Il **Type Checking** (presente per esempio in TypeScript) assicura il corretto svolgimento delle operazioni.

Inoltre, il type checking rende il programma più efficiente ed evita errori nelle operazioni. Si dice che il type checking offra una **correttezza parziale** (solo per alcuni tipi di errori) ed è un esempio di tecnica di analisi di programma (statica e dinamica).

5 Linguaggi

5.1 Fortran

Fortran (FORMula TRANsfer) è uno dei linguaggi più antichi, realizzato da **John Backus** (1924-2007) negli anni 50 all'IBM, primo linguaggio di alto livello implementato efficacemente (memoria statica, senza annidamento).

Ha introdotto le variabili come le conosciamo oggi, i cicli, le procedure, le etichette delle istruzioni e molto altro.

Le prime versioni non supportavano la ricorsione e includevano caratteristiche oggi abbandonate.

5.2 ALGOL 60

ALGOL (ALGOritmic Language) fu progettato nel 1958-1960 come linguaggio universale.

Ha introdotto innovazioni fondamentali:

- Sintassi formalmente specificata (BNF)
- Scoping lessicale / blocchi
- Procedure modulari
- Passaggi per nome
- Procedure ricorsive
- Dichiarazioni di tipi di variabili
- Allocazione dello stack e record di attivazione

5.3 LISP

Il **LISP** (LISt Processor) è un linguaggio funzionale, sviluppato da **John McCarty** nel 1958 al MIT per la manipolazione di liste.

Precursore di molti concetti, tra cui la gestione dinamica della memoria (heap e garbage collector).

5.4 SIMULA 67

SIMULA 67 è stato sviluppato da **Ole-Johan** e **Kristen Nygaard**, è un'estensione di ALGOL 60.

Fu il primo linguaggio orientato agli oggetti, con classi, oggetti, ereditarietà e sottoclassi. Introdusse anche passaggi per valore e riferimento, puntatori e **co-routine** (simili ai thread).

5.5 C

Introdotta nei Bell Labs 1972 da **Ritchie**, il suo sviluppo è strettamente legato a **UNIX** e ai sistemi operativi.

Maggiori possibilità di accessi alle funzionalità di basso livello della macchina.

Gestione manuale della memoria (puntatori ma no garbage collection). Le funzioni non si possono annidare.

Controllo dei tipi non forte.

Linguaggio imperativo.

5.6 Pascal

progettato da **N. Wirth** agli inizi degli anni '70, prende il nome dal filosofo **Blaise Pascal**, inventore della **pascalina**, strumento di calcolo precursore della calcolatrice moderna. Rappresenta una versione più semplice di ALGOL 60.

Linguaggio fortemente tipizzato, gestione dinamica della memoria e scoping statico.

Offre molti tipi di dati (anche custom) come array, record, file e set.

Supporta le funzioni.

5.7 Smalltalk

Introdotta negli anni '70 da **Alan Kay** allo Xerox Palo Alto Research Center per supportare nozioni di incapsulamento e information hiding.

Tutto è un oggetto tranne le variabili; la computazione avviene tramite uno scambio di messaggi.

Utilizza il garbage collector ed è dotato di GUI e IDE integrati.

5.8 PROLOG

PROLOG (PROgrammation en LOGique) fu introdotta da **Robert Kowalski** (teoria), **Marten Van Emden** (dimostrazione sperimentale) e implementata finalmente da **Alain Colmerauer** all'inizio degli anni '70.

Linguaggio dichiarativo

5.9 C++

Introdotta nei Bell Labs nel 1986 da **Bjarne Stroustrup**.

Gli oggetti sono una generalizzazione di "struct".

Estende **C** con le classi ed un migliore sistema di tipi.

Supporta concetti complessi come l'ereditarietà multipla, modelli / generici (template) e la gestione delle eccezioni.

5.10 Java

Introdotta alla SUN 1991-1995 dal gruppo di **James Gosling**, completamente orientato agli oggetti.

Ha garbage collection e quindi la gestione della memoria non è esplicita; non c'è ereditarietà multipla e neanche generics.

Si possono usare thread in maniera concorrente.

Presta molto attenzione alla portabilità dato il suo aspetto ibrido tra linguaggio compilato e interpretato.

5.11 Altri linguaggi popolari

- **Imperativi:** Modula, Oberon, Ada
- **Funzionali:** Miranda, Haskell, ML, Caml, OCaml, Scheme, Common LISP
- **Orientati agli oggetti:** Objective-C, Eiffel, Modula-3, Self, C#, CLOS
- **Logici:** Gödel, LDL, ACL2, Isabelle, HOL
- **Scripting:** (Text processing) Perl, Python; (Web programming) JavaScript, PHP
- **Elaborazione dati e database:** Cobol, SQL, 4GL, XQuery
- **Programmazione di sistemi:** PL/I, PL/M, BLISS
- **Applicazioni specializzate:** APL, Forth, Icon, Logo, SNOBOL4, GPSS, Visual Basic
- **Concorrenti:** Concurrent Pascal, Concurrent C, C*, SR, Occam, Erlang, Obliq

5.12 Come crescono i linguaggi?

Ci sono diversi fattori che fanno crescere un linguaggio, eccone diversi:

- **Killer App:** un linguaggio è la scelta gettonata per sviluppare app di successo (Java, PHP, R)
- **Esclusività delle piattaforme:** alcuni linguaggi si impongono perchè vincolati a un ecosistema (Swift e iOS).
- **Velocità dell'upgrade:** evoluzione rapida che mantiene la semplicità (Python)
- **Grande slogan:** fa la differenza (Java: write once, run everywhere)
- **Slow and steady:** crescita lenta ma stabile e duratura (C)
- **Sintassi chiara:** abbassa la barriera d'ingresso (Python)
- **Job market:** orienta le scelte (Java, JavaScript, Python)
- **Community:** il contributo degli appassionati tramite numerosissime librerie (JavaScript, Python)

6 Approccio scientifico

Per studiare i linguaggi di programmazione si attuerà un **metodo scientifico**:

1. **Fatti:** la realtà che va studiata (osservazioni, esperimenti, dati...)
2. **Modello:** la rappresentazione tramite leggi matematiche.
3. **Ragionamento:** analisi della realtà attraverso il modello.

Questo metodo viene applicato in tutti i campi della scienza e ricerca e lo applicheremo anche noi in questo modo:

1. **Fatti:** linguaggi di programmazione.
2. **Modello:** semantica.
3. **Ragionamento:** comprensione e studio di proprietà.

7 Lambda Calculus

Il **lambda calcolo** può essere considerato il primo linguaggio di programmazione funzionale, utilizza questa segnatura:

$$\lambda x.x$$

- λ = introduce la funzionale
- x = parametro formale (1); corpo (2)???

7.1 Storia della calcolabilità

7.2 Macchina di Turing

Con la macchina di Turing si possono rappresentare tutti gli algoritmi in esistenza. Modelli di calcolo meno potenti:

- Espressioni regolari
- Funzioni che terminano sempre (funzioni totali)

7.3 Architettura di Von Neumann